

PYTHON TUTORIAL

Even Numbers File Filter

A step-by-step guide to reading files, filtering data,
and writing output in Python.

Prepared for Python Learners

 Python Logo Minimalist

The Challenge

Goal

Write a Python program that reads a file named `numbers.txt` containing integers (one per line) and creates a new file `even.txt`.

- ✓ Only **even numbers** should be written.
- ✓ Maintain one number per line.
- ✓ **Constraint:** The program must run without raising type errors.



Input vs. Output

numbers.txt (Input)

1
2
3
4
5
6



Filter Logic

even.txt (Output)

2
4
6

We only keep integers where $x \% 2 = 0$

Key Python Concepts



File I/O

`open('file', 'r')` to read.
`open('file', 'w')` to write.

Always close files!



Modulo Operator

The `%` operator returns the remainder.

`number % 2 == 0` checks for evenness.



Type Conversion

Files are read as strings.
Use `int()` to convert text to numbers for calculation.

Solution 1: The "Iterative" Approach

How it works

We read the input file one line at a time. For each line, we immediately check if it's even and write it to the output file if it is.

Pros

- ✓ **Memory Efficient:** Only holds one line in memory at a time. Great for huge files.
- ✓ **Simple Logic:** Easy to follow step-by-step.



Line-by-Line Processing

Flowchart: Iterative Approach



Open Files

Input (Read) & Output
(Write)



Read Line

Get next line from file



Check Even

Is `int(line) % 2 == 0`?



Write

If even, write to output



Repeat

Until end of file

Code: Iterative Solution

```
def filter_even_numbers():
    try:
        with open('numbers.txt', 'r') as infile, \
            open('even.txt', 'w') as outfile:

            for line in infile:
                try:
                    num = int(line.strip())
                    if num % 2 == 0:
                        outfile.write(f"{num}\n")
                except ValueError:
                    pass # Skip invalid lines

    except FileNotFoundError:
        print("Input file not found.")

filter_even_numbers()
```

Code Breakdown

- `with open( ... )`: Automatically closes files even if errors occur.
- `for line in infile`: Loops through the file efficiently.
- `strip()`: Removes hidden newline characters before conversion.
- `try-except`: Handles non-integer lines safely.

Solution 2: The "List Comprehension" Approach

How it works

We read the entire file content into a list first. Then, we use Python's powerful **list comprehension** syntax to filter the data in a single, readable line of code.

Pros

- ✓ **Concise:** Much less code to write.
- ✓ **Pythonic:** Uses modern Python features.
- ✓ **Fast:** Good for small to medium files.



Flowchart: List Comprehension



1. Read All

Load all lines from
`numbers.txt` into a memory
list at once.



2. Filter List

Create a new list containing
only valid even integers
using logic.



3. Write Block

Write the entire filtered list to
`even.txt` in one operation.

Code: List Comprehension

```
def filter_even_fast():
    try:
        with open('numbers.txt', 'r') as f:
            lines = f.readlines()

        # Filter logic in one line
        evens = [
            line for line in lines
            if line.strip().isdigit()
            and int(line) % 2 == 0
        ]

        with open('even.txt', 'w') as f:
            f.writelines(evens)

    except FileNotFoundError:
        print("File not found")
```

Code Breakdown

- `readlines()`: Returns a list of strings (lines).
- `[x for x in list if condition]`: Creates a new list based on criteria.
- `isdigit()`: A simple check to ensure the line contains a number (basic error check).
- `writelines()`: Writes a list of strings to a file efficiently.

Comparing Approaches

Feature	Solution 1 (Iterative)	Solution 2 (List Comp)
Memory Usage	Low (One line at a time)	High (Loads full file)
Code Length	More verbose	Concise
Performance	Better for large files	Faster for small files
Readability	Clear logic flow	"Pythonic" & compact

Handling Errors

Avoiding Crashes

The problem states: *"run without raising any type errors"*.

If the input file contains text like "hello" instead of a number, a standard `int()` conversion will crash the program.

The Fix: Wrap the conversion in a `try-except` `ValueError` block.

Dangerous Input:

```
1
2
banana <-- Crash!
4
```

Safe Code:

```
try:
    num = int(line)
except ValueError:
    continue # Safe skip
```


Formatting Details

Why `.strip()`?

- ✓ Lines read from a file usually end with a hidden `\n` (newline) character.
- ✓ `"4\n"` is not the same as `"4"`. Converting `"4\n"` works, but spaces around it might cause issues. `strip()` cleans it up.

Why add `\n` when writing?

- ✓ `write()` does **not** add a new line automatically.
- ✓ If you write 2 then 4, you get 24.
- ✓ You must explicitly write `f"{num}\n"` or `str(num) + '\n'` to get:
2
4